

Phase 1 Quick Start Implementation Guide

This document provides concrete code templates to jump-start Phase 1 implementation.

1. Experiment Runner Implementation

File: `benchmark/experiments/runner.py`

```
"""
Experiment runner for blood glucose forecasting benchmark.
"""

import yaml
import json
from pathlib import Path
from typing import Dict, List, Optional, Any
from datetime import datetime

# Import from your existing components
from ..configs.config_manager import ConfigManager
from ..evaluation.evaluator import BGEvaluator
from .transfer_learning_experiment import TransferLearningExperiment

class ExperimentRunner:
    """Main experiment runner that orchestrates benchmark
    experiments."""

    def __init__(self, config_path: Optional[str] = None, config_dict:
Optional[Dict] = None):
        """
        Initialize experiment runner.

        Args:
            config_path: Path to YAML configuration file
            config_dict: Configuration dictionary (alternative to file)
        """
        self.config_manager = ConfigManager()

        if config_path:
            self.config = self.config_manager.load_config(config_path)
        elif config_dict:
            self.config = config_dict
        else:
            raise ValueError("Either config_path or config_dict must be
provided")

        # Validate configuration
        self.config_manager.validate_config(self.config)
```

```

        self.experiment_type = self.config.get('experiment',
{}).get('type', 'standard')
        self.results_dir = Path(self.config.get('output',
{}).get('results_dir', 'benchmark/results/experiments'))
        self.results_dir.mkdir(parents=True, exist_ok=True)

    def run_experiment(self) -> Dict[str, Any]:
        """
        Run a single experiment based on configuration.

        Returns:
            Dictionary containing experiment results
        """
        print(f"🚀 Starting experiment: {self.config['experiment']
['name']}")
        print(f"🇫🇷 Experiment type: {self.experiment_type}")

        # Create experiment instance based on type
        if self.experiment_type == 'transfer_learning':
            experiment = TransferLearningExperiment(self.config)
        else:
            raise ValueError(f"Unsupported experiment type:
{self.experiment_type}")

        # Run experiment
        results = experiment.run()

        # Save results
        self._save_experiment_results(results)

        print("✅ Experiment completed successfully!")
        return results

    def run_batch_experiments(self, config_dir: str) -> List[Dict[str,
Any]]:
        """
        Run multiple experiments from configuration directory.

        Args:
            config_dir: Directory containing YAML config files

        Returns:
            List of experiment result dictionaries
        """
        config_path = Path(config_dir)
        config_files = list(config_path.glob("*.yaml")) +
list(config_path.glob("*.yml"))

        if not config_files:
            raise ValueError(f"No YAML config files found in
{config_dir}")

        print(f"🔄 Running batch experiments: {len(config_files)}")

```

```

configs found")

    batch_results = []
    for config_file in config_files:
        print(f"\n{'='*50}")
        print(f"Running experiment: {config_file.name}")
        print(f"{'='*50}")

        try:
            runner = ExperimentRunner(config_path=str(config_file))
            results = runner.run_experiment()
            batch_results.append(results)
        except Exception as e:
            print(f"❌ Failed to run {config_file.name}: {e}")
            continue

    # Save batch summary
    self._save_batch_summary(batch_results)

    return batch_results

def _save_experiment_results(self, results: Dict[str, Any]) -> None:
    """Save experiment results to timestamped directory."""
    timestamp = datetime.now().strftime('%Y%m%d_%H%M%S')
    experiment_name = self.config['experiment']['name']

    experiment_dir = self.results_dir / f"
{timestamp}_{experiment_name}"
    experiment_dir.mkdir(parents=True, exist_ok=True)

    # Save configuration
    with open(experiment_dir / "config.yaml", 'w') as f:
        yaml.dump(self.config, f, default_flow_style=False)

    # Save results
    with open(experiment_dir / "results.json", 'w') as f:
        json.dump(results, f, indent=2, default=str)

    # Save summary
    self._create_experiment_summary(results, experiment_dir)

    print(f"📁 Results saved to: {experiment_dir}")

def _save_batch_summary(self, batch_results: List[Dict]) -> None:
    """Save summary of batch experiment results."""
    timestamp = datetime.now().strftime('%Y%m%d_%H%M%S')
    batch_dir = self.results_dir / f"batch_{timestamp}"
    batch_dir.mkdir(parents=True, exist_ok=True)

    # Create summary
    summary = {
        'batch_timestamp': timestamp,
        'total_experiments': len(batch_results),

```

```

        'successful_experiments': len([r for r in batch_results if
r.get('status') == 'success']),
        'experiments': batch_results
    }

    with open(batch_dir / "batch_summary.json", 'w') as f:
        json.dump(summary, f, indent=2, default=str)

    print(f"🇫🇷 Batch summary saved to: {batch_dir}")

    def _create_experiment_summary(self, results: Dict, save_dir: Path)
-> None:
        """Create human-readable experiment summary."""
        summary_lines = [
            f"# Experiment Summary: {self.config['experiment']
['name']}",
            f"*Date*: {datetime.now().strftime('%Y-%m-%d %H:%M:%S')}",
            f"*Type*: {self.experiment_type}",
            "",
            "## Configuration",
            f"- Dataset: {self.config.get('data', {}).get('dataset',
'N/A')}",
            f"- Model: {self.config.get('model', {}).get('type',
'N/A')}",
            f"- Patients: {self.config.get('data', {}).get('subjects',
'N/A')}",
            "",
            "## Results Summary"
        ]

        # Add key metrics if available
        if 'model_comparison' in results:
            summary_lines.append("### Model Performance")
            for patient_id, patient_results in
results['model_comparison'].items():
                summary_lines.append(f"\n**Patient {patient_id}:**")
                for model_name, metrics in patient_results.items():
                    mae = metrics.get('mae', 'N/A')
                    mard = metrics.get('mard', 'N/A')
                    summary_lines.append(f"- {model_name}: MAE=
{mae:.2f}, MARD={mard:.1f}%")

            with open(save_dir / "summary.md", 'w') as f:
                f.write('\n'.join(summary_lines))

# Convenience functions for CLI
def run_single_experiment(config_path: str) -> Dict[str, Any]:
    """Run a single experiment from config file."""
    runner = ExperimentRunner(config_path=config_path)
    return runner.run_experiment()

def run_batch_experiments(config_dir: str) -> List[Dict[str, Any]]:

```

```

        """Run batch experiments from config directory."""
        runner = ExperimentRunner(config_dict={'experiment': {'name':
'batch_run', 'type': 'batch'}})
        return runner.run_batch_experiments(config_dir)

```

2. Configuration Manager Implementation

File: `benchmark/configs/config_manager.py`

```

"""
Configuration management utilities for the benchmark.
"""

import yaml
from pathlib import Path
from typing import Dict, Any, List
import jsonschema
from jsonschema import validate, ValidationError

class ConfigManager:
    """Handles loading, validation, and merging of experiment
    configurations."""

    def __init__(self):
        self.config_dir = Path(__file__).parent
        self.schema = self._load_config_schema()

    def load_config(self, config_path: str) -> Dict[str, Any]:
        """
        Load configuration from YAML file.

        Args:
            config_path: Path to configuration file

        Returns:
            Configuration dictionary
        """
        config_path = Path(config_path)

        if not config_path.exists():
            raise FileNotFoundError(f"Configuration file not found:
{config_path}")

        with open(config_path, 'r') as f:
            config = yaml.safe_load(f)

        # Merge with default config
        default_config = self._load_default_config()
        merged_config = self._merge_configs(default_config, config)

```

```

        return merged_config

def validate_config(self, config: Dict[str, Any]) -> bool:
    """
    Validate configuration against schema.

    Args:
        config: Configuration dictionary

    Returns:
        True if valid, raises ValidationError if invalid
    """
    try:
        validate(instance=config, schema=self.schema)
        return True
    except ValidationError as e:
        raise ValueError(f"Configuration validation failed: {e.message}")

def _load_default_config(self) -> Dict[str, Any]:
    """Load default configuration."""
    default_path = self.config_dir / "default.yaml"

    if not default_path.exists():
        # Return minimal default if file doesn't exist
        return {
            'experiment': {'name': 'unnamed_experiment', 'type':
'standard'},
            'data': {'dataset': 'ohiot1dm', 'subjects': 'all'},
            'model': {'type': 'lstm'},
            'evaluation': {'metrics': ['mae', 'rmse', 'mard']},
            'output': {'save_model': True, 'save_predictions': True}
        }

    with open(default_path, 'r') as f:
        return yaml.safe_load(f)

def _merge_configs(self, default: Dict, user: Dict) -> Dict:
    """Recursively merge user config with default config."""
    result = default.copy()

    for key, value in user.items():
        if key in result and isinstance(result[key], dict) and
isinstance(value, dict):
            result[key] = self._merge_configs(result[key], value)
        else:
            result[key] = value

    return result

def _load_config_schema(self) -> Dict[str, Any]:
    """Load JSON schema for configuration validation."""
    return {

```

```

        "type": "object",
        "properties": {
            "experiment": {
                "type": "object",
                "properties": {
                    "name": {"type": "string"},
                    "type": {"type": "string", "enum": ["standard",
"transfer_learning", "cross_validation"]},
                    "description": {"type": "string"},
                    "author": {"type": "string"}
                },
                "required": ["name", "type"]
            },
            "data": {
                "type": "object",
                "properties": {
                    "dataset": {"type": "string"},
                    "subjects": {"oneOf": [{"type": "string"},
{"type": "array"}]},
                    "train_ratio": {"type": "number", "minimum": 0,
"maximum": 1},
                    "val_ratio": {"type": "number", "minimum": 0,
"maximum": 1},
                    "test_ratio": {"type": "number", "minimum": 0,
"maximum": 1}
                },
                "required": ["dataset"]
            },
            "model": {
                "type": "object",
                "properties": {
                    "type": {"type": "string"},
                    "hyperparameters": {"type": "object"}
                },
                "required": ["type"]
            },
            "evaluation": {
                "type": "object",
                "properties": {
                    "metrics": {
                        "type": "array",
                        "items": {"type": "string"}
                    }
                }
            }
        },
        "required": ["experiment", "data", "model"]
    }

```

```

def load_config(config_path: str) -> Dict[str, Any]:
    """Convenience function to load and validate config."""
    manager = ConfigManager()
    return manager.load_config(config_path)

```

3. Transfer Learning Experiment Class

File: `benchmark/experiments/transfer_learning_experiment.py`

```
"""
Transfer learning experiment implementation.
"""

from typing import Dict, Any, List
import sys
from pathlib import Path

# Add benchmark to path for imports
sys.path.append(str(Path(__file__).parent.parent.parent))

from benchmark.data.loaders import load_ohiot1dm_data
from benchmark.data.preprocessors import preprocess_ohiot1dm_data
from benchmark.data.torch_dataset import prepare_multi_patient_dataset
from benchmark.models.rnn import RNNBGModel, LSTMBGModel, GRUBGModel
from benchmark.evaluation.evaluator import BGEvaluator

class TransferLearningExperiment:
    """Transfer learning experiment that integrates with your existing
    code."""

    def __init__(self, config: Dict[str, Any]):
        self.config = config
        self.data_config = config.get('data', {})
        self.model_config = config.get('model', {})
        self.experiment_config = config.get('experiment', {})

        # Initialize evaluator
        self.evaluator = BGEvaluator(
            default_metrics=config.get('evaluation', {}).get('metrics',
            ['mae', 'rmse', 'mard', 'tir']),
        )

        # Model classes
        self.model_classes = {
            'rnn': RNNBGModel,
            'lstm': LSTMBGModel,
            'gru': GRUBGModel
        }

    def run(self) -> Dict[str, Any]:
        """
        Run the transfer learning experiment.

        Returns:

```

```

        Dictionary containing experiment results
    """
    try:
        # Load and prepare data
        patient_data = self._load_and_prepare_data()

        # Run model training and evaluation
        model_results = self._run_model_comparison(patient_data)

        # Compile results
        results = {
            'experiment_info': {
                'name': self.experiment_config.get('name'),
                'type': 'transfer_learning',
                'status': 'success'
            },
            'configuration': self.config,
            'model_comparison': model_results,
            'summary': self._generate_summary(model_results)
        }

        return results

    except Exception as e:
        return {
            'experiment_info': {
                'name': self.experiment_config.get('name'),
                'type': 'transfer_learning',
                'status': 'failed',
                'error': str(e)
            },
            'configuration': self.config
        }

def _load_and_prepare_data(self) -> Dict[str, Any]:
    """Load and prepare data using existing pipeline."""
    # Extract parameters from config
    patient_ids = self.data_config.get('subjects', [540, 544])
    if isinstance(patient_ids, str) and patient_ids == 'all':
        patient_ids = [540, 544, 552, 567, 584, 596] # Default
        OhioT1DM patients

    version = self.data_config.get('version', '2020')
    sequence_length = self.data_config.get('sequence_length', 12)
    prediction_horizon = self.data_config.get('prediction_horizon',

6)
    unimodal = self.data_config.get('unimodal', False)

    # Use your existing data loading logic
    # This would call the same functions as in run_bg_models.py
    # but through the benchmark framework

    return {

```

```

        'patient_ids': patient_ids,
        'sequence_length': sequence_length,
        'prediction_horizon': prediction_horizon,
        'unimodal': unimodal
    }

    def _run_model_comparison(self, patient_data: Dict[str, Any]) ->
Dict[str, Any]:
        """Run model comparison using existing logic."""
        # This would integrate your existing transfer learning logic
        # from run_bg_models.py into the benchmark framework

        model_type = self.model_config.get('type', 'lstm')

        # Placeholder - integrate with your existing BGModelTrainer
logic
        results = {
            'model_type': model_type,
            'patients_evaluated': patient_data['patient_ids'],
            'metrics': 'placeholder - integrate with existing
evaluation'
        }

        return results

    def _generate_summary(self, model_results: Dict[str, Any]) ->
Dict[str, Any]:
        """Generate experiment summary."""
        return {
            'total_patients':
len(model_results.get('patients_evaluated', [])),
            'model_type': model_results.get('model_type'),
            'status': 'completed'
        }

```

4. CLI Updates

File: `benchmark/cli.py`

```

"""
Command Line Interface for the benchmark.
"""

import argparse
from pathlib import Path
from .experiments.runner import ExperimentRunner, run_single_experiment,
run_batch_experiments

def create_cli_parser():
    """Create the CLI argument parser."""

```

```

    parser = argparse.ArgumentParser(description="Blood Glucose
Forecasting Benchmark")
    subparsers = parser.add_subparsers(dest='command', help='Available
commands')

    # Run single experiment
    run_parser = subparsers.add_parser('run', help='Run a single
experiment')
    run_parser.add_argument('--config', required=True, help='Path to
experiment configuration file')

    # Run batch experiments
    batch_parser = subparsers.add_parser('batch', help='Run batch
experiments')
    batch_parser.add_argument('--config-dir', required=True,
help='Directory containing config files')

    # Validate configuration
    validate_parser = subparsers.add_parser('validate', help='Validate
configuration file')
    validate_parser.add_argument('--config', required=True, help='Path
to configuration file')

    # Legacy mode (backward compatibility with run_bg_models.py)
    legacy_parser = subparsers.add_parser('legacy', help='Run in legacy
mode (like run_bg_models.py)')
    legacy_parser.add_argument('--patients', nargs='+', type=int,
default=[540, 544])
    legacy_parser.add_argument('--pretrain-epochs', type=int,
default=50)
    legacy_parser.add_argument('--finetune-epochs', type=int,
default=25)

    return parser

def main():
    """Main CLI entry point."""
    parser = create_cli_parser()
    args = parser.parse_args()

    if args.command == 'run':
        results = run_single_experiment(args.config)
        print(f"✅ Experiment completed. Results:
{results['experiment_info']['status']}")

    elif args.command == 'batch':
        results = run_batch_experiments(args.config_dir)
        print(f"✅ Batch completed. {len(results)} experiments run.")

    elif args.command == 'validate':
        from .configs.config_manager import ConfigManager
        manager = ConfigManager()
        try:

```

```

        config = manager.load_config(args.config)
        manager.validate_config(config)
        print("✅ Configuration is valid")
    except Exception as e:
        print(f"❌ Configuration validation failed: {e}")

    elif args.command == 'legacy':
        # Call your existing run_bg_models.py logic
        print("🔄 Running in legacy mode...")
        print("📖 This will be replaced with benchmark framework integration")

    else:
        parser.print_help()

if __name__ == '__main__':
    main()

```

5. Sample Transfer Learning Configuration

File: `benchmark/configs/transfer_learning_example.yaml`

```

experiment:
  name: "transfer_learning_lstm_example"
  type: "transfer_learning"
  description: "LSTM with transfer learning on OhioT1DM dataset"
  author: "Your Name"

data:
  dataset: "ohiot1dm"
  subjects: [540, 544] # or "all" for all patients
  version: "2020"
  sequence_length: 12
  prediction_horizon: 6
  unimodal: false

model:
  type: "lstm"
  hyperparameters:
    hidden_size: 64
    num_layers: 2
    dropout: 0.2
    learning_rate: 0.001

transfer_learning:
  pretrain_epochs: 50
  finetune_epochs: 25
  early_stopping_patience: 15

evaluation:

```

```
metrics:
  - "mae"
  - "rmse"
  - "mard"
  - "tir"
  - "clarke"

output:
  results_dir: "benchmark/results/experiments"
  save_model: true
  save_predictions: true
  generate_plots: true
```

This gives you a concrete starting point for Phase 1 implementation. The key is to gradually integrate your existing working code from `run_bg_models.py` into this framework while maintaining functionality.